

DDA5001 Machine Learning

Linear Classification III: Support Vector Machine (SVM)

Yuning You

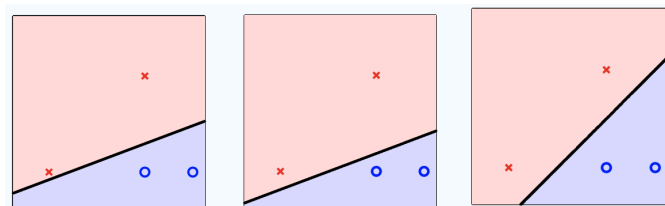
School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Agenda

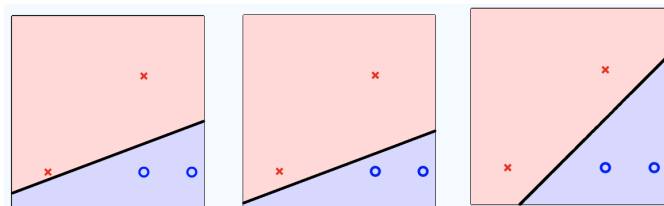
Support Vector Machine (SVM)

Perceptron: Which is The Optimal Classifier?



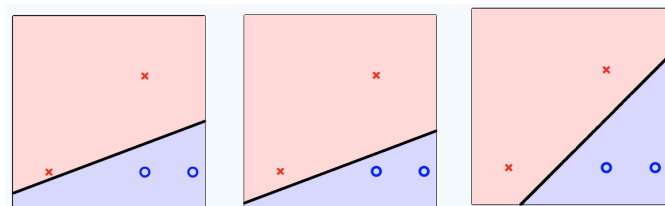
- Which classifier is possibly generated by perceptron?

Perceptron: Which is The Optimal Classifier?



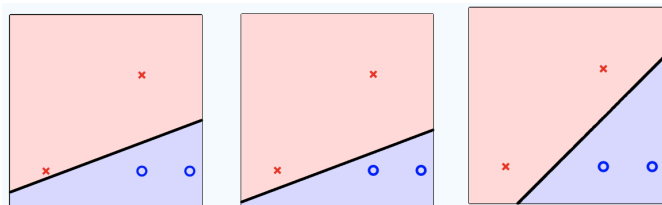
- Which classifier is possibly generated by perceptron? All three.

Perceptron: Which is The Optimal Classifier?



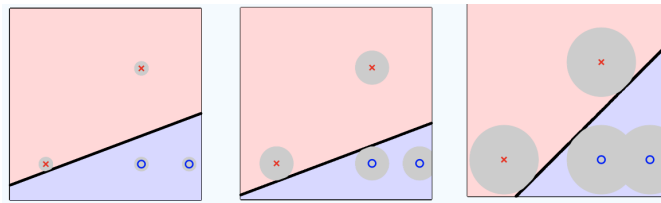
- ▶ Which classifier is possibly generated by perceptron? All three.
- ▶ Apart from the above three lines, the perceptron learned model can actually have **infinitely many** possibilities once it classifies the training data.
- ▶ From VC analysis perspective, all the classifiers provide $Er_{in} = 0$, and hence we have the **same Er_{out} -bound** for every classifier. VC analysis is a **worst-case result for any classifier**, it does not account for some other factors (we will specify in the next slide).

Perceptron: Which is The Optimal Classifier?



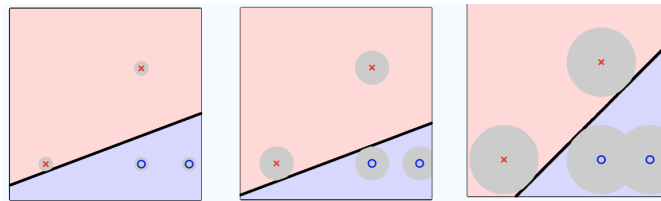
- ▶ Which classifier is possibly generated by perceptron? All three.
- ▶ Apart from the above three lines, the perceptron learned model can actually have **infinitely many** possibilities once it classifies the training data.
- ▶ From VC analysis perspective, all the classifiers provide $Er_{in} = 0$, and hence we have the **same Er_{out} -bound** for every classifier. VC analysis is a **worst-case result for any classifier**, it does not account for some other factors (we will specify in the next slide).
- ▶ **Intuitively, the third classifier is the best.**

Noise



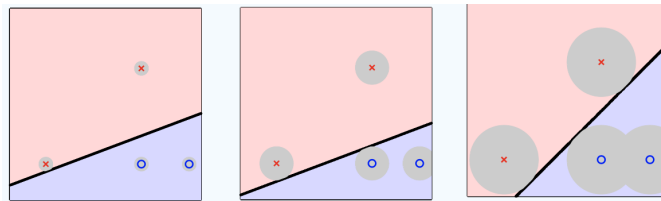
- In practice, there are measurement errors – **noise** (stochastic). The noise effect is illustrated as shaded area around each point. The true data point can lie anywhere within this ‘region of uncertainty’ due to the measurement error.

Noise



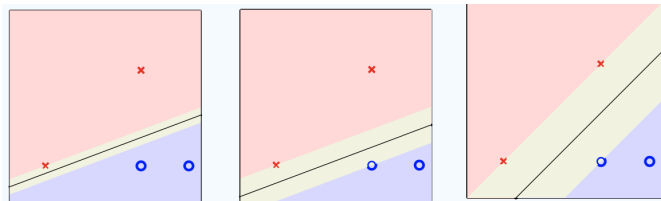
- ▶ In practice, there are measurement errors – **noise** (stochastic). The noise effect is illustrated as shaded area around each point. The true data point can lie anywhere within this ‘region of uncertainty’ due to the measurement error.
- ▶ A classifier is **robust** to the noise if it can classify the **true data points** correctly.
- ▶ A classifier that can **tolerate more noise**, quantified as the radius of the shaded area, is **more robust**.

Noise



- ▶ In practice, there are measurement errors – **noise** (stochastic). The noise effect is illustrated as shaded area around each point. The true data point can lie anywhere within this ‘region of uncertainty’ due to the measurement error.
- ▶ A classifier is **robust** to the noise if it can classify the **true data points** correctly.
- ▶ A classifier that can **tolerate more noise**, quantified as the radius of the shaded area, is **more robust**.
- ▶ In overfitting section, we know that noise is one of the main causes of overfitting. Regularization helps us combat noise and avoid overfitting. In our example, the rightmost classifier is more robust to noise without compromising Er_{in} . **It is better ‘regularized’**.

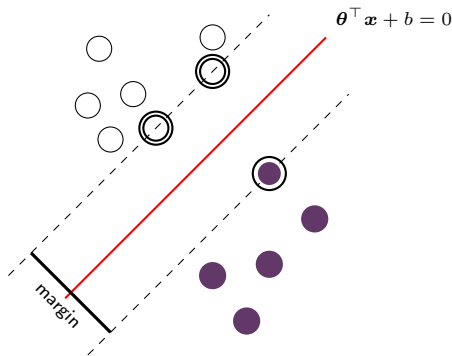
Quantify Noise Tolerance From The Classifier



- ▶ We can draw the **separation region** (yellow region) of the classifier that is formed by the area between two classes. The thickness reflects the amount of noise the classifier can tolerate. If any data point is perturbed by at most the thickness of each side, it will still be on the correct side of the classifier.
- ▶ The **maximum thickness** (noise tolerance) for a classifier is called its **margin**.

Maximal Margin Hyperplane

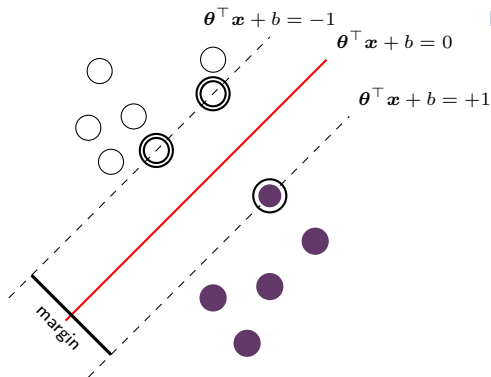
Support Vectors and Maximal Margin



- ▶ Select two **parallel** hyperplanes that separate the two classes and the distance between them is **as large as possible**.
- ▶ **Support vectors** determines these two parallel hyperplanes.
- ▶ **Margin** is the distance bounded by these two hyperplanes.
- ▶ The **maximum-margin hyperplane** lies halfway between them, we define it as

$$\theta^T x + b = 0$$

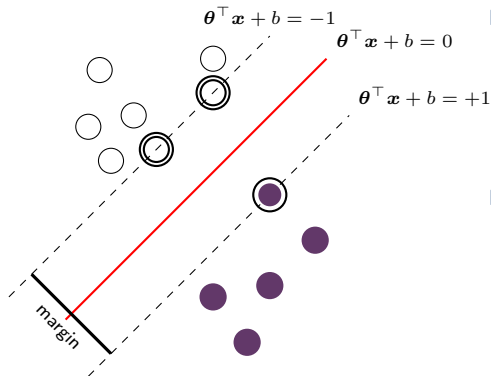
Expression of The Two Parallel Hyperplane



- ▶ Denote $\{x^s\}$ as the support vectors.
- ▶ The two parallel hyperplane passing the support vectors should have the expression:

$$\theta^\top x^s + b = \pm c.$$

Expression of The Two Parallel Hyperplane



- ▶ Denote $\{x^s\}$ as the support vectors.
- ▶ The two parallel hyperplane passing the support vectors should have the expression:

$$\theta^\top x^s + b = \pm c.$$

- ▶ We can apply normalization to θ and b to get

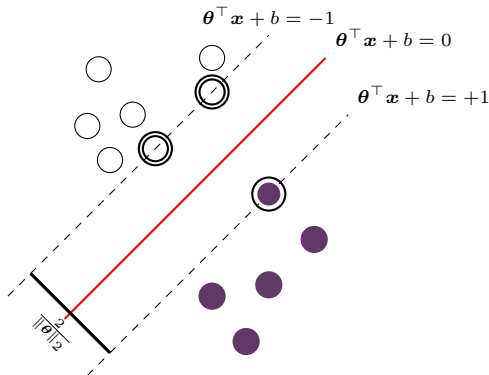
$$\theta^\top x^s + b = 1.$$

and

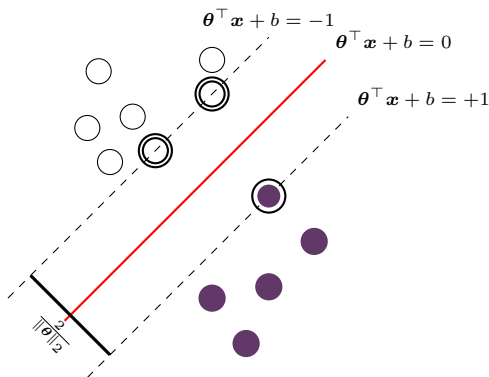
$$\theta^\top x^s + b = -1.$$

Expression of The Margin

- ▶ Margin = $2 \times$ distance between $\mathbf{x}^s$ and the maximum-margin hyperplane.
- ▶ Margin also equals to the distance between two parallel hyperplanes: $\boldsymbol{\theta}^\top \mathbf{x}^s + b = 1$ and $\boldsymbol{\theta}^\top \mathbf{x}^s + b = -1$.



Expression of The Margin



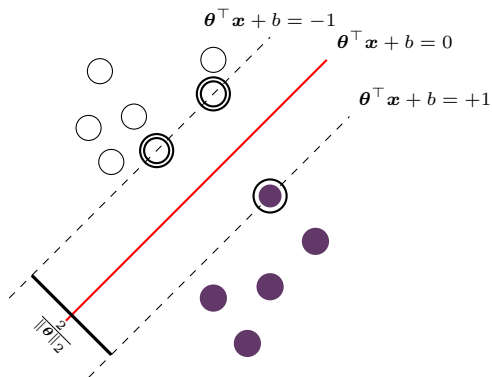
- ▶ Margin = $2 \times$ distance between x^s and the maximum-margin hyperplane.
- ▶ Margin also equals to the distance between two parallel hyperplanes: $\theta^\top x^s + b = 1$ and $\theta^\top x^s + b = -1$.
- ▶ Fact: The distance between two parallel hyperplanes, $\theta^\top x^s + b = a_1$ and $\theta^\top x^s + b = a_2$, is given by

$$\frac{|a_1 - a_2|}{\|\theta\|_2}.$$

- ▶ Thus, The margin:

$$\frac{2}{\|\theta\|_2}$$

The Learning Problem



The principle: Maximize the margin

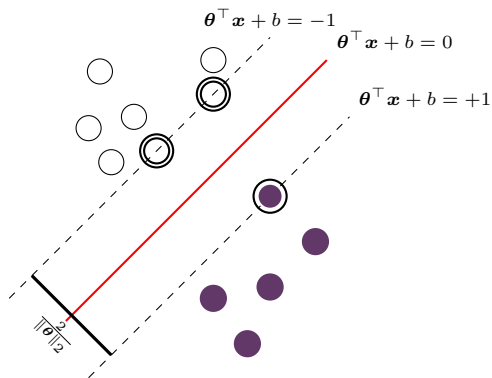
$$\underset{\theta \in \mathbb{R}^d}{\text{maximize}} \quad \frac{2}{\|\theta\|_2},$$

which is equivalent to

$$\underset{\theta \in \mathbb{R}^d}{\text{minimize}} \quad \|\theta\|_2^2.$$

Is it done?

The Learning Problem



We forgot the constraint: $|\theta^\top x^s + b| = 1$. Since we often do not know which data point is a support vector as a prior, we need to use all the data points. Thus, we have

$$\underset{\theta \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \|\theta\|_2^2$$

$$\text{subject to} \quad |\theta^\top x_i + b| \geq 1, \quad i = 1, \dots, n$$

Hard-margin SVM.

Note that $|\boldsymbol{\theta}^\top \mathbf{x}_i + b| = y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$.

We finally have the optimization formulation

$$\begin{array}{ll}\text{minimize} & \|\boldsymbol{\theta}\|_2^2 \\ \boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R} & \\ \text{subject to} & y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n\end{array}$$

This is called **hard-margin support vector machine (SVM)**.

Hard-margin SVM.

Note that $|\boldsymbol{\theta}^\top \mathbf{x}_i + b| = y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$.

We finally have the optimization formulation

$$\begin{array}{ll}\text{minimize} & \|\boldsymbol{\theta}\|_2^2 \\ \boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R} & \\ \text{subject to} & y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n\end{array}$$

This is called **hard-margin support vector machine (SVM)**.

- ▶ The word “hard-margin” means that all **training** data points must be correctly classified.
- ▶ This is ensured by the constraints, and can be regarded as a perceptron classifier.

Hard-margin SVM.

Note that $|\boldsymbol{\theta}^\top \mathbf{x}_i + b| = y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$.

We finally have the optimization formulation

$$\begin{array}{ll}\text{minimize} & \|\boldsymbol{\theta}\|_2^2 \\ \boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R} & \\ \text{subject to} & y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n\end{array}$$

This is called **hard-margin support vector machine (SVM)**.

- ▶ The word “hard-margin” means that all **training** data points must be correctly classified.
- ▶ This is ensured by the constraints, and can be regarded as a perceptron classifier.
- ▶ Thus, hard-margin SVM can be regarded as: **Choose a classifier from all possible perceptron classifiers that has the largest margin.**

Hard-margin SVM.

Note that $|\boldsymbol{\theta}^\top \mathbf{x}_i + b| = y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)$.

We finally have the optimization formulation

$$\begin{aligned} & \underset{\boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \|\boldsymbol{\theta}\|_2^2 \\ & \text{subject to} \quad y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

This is called **hard-margin support vector machine (SVM)**.

- ▶ The word “hard-margin” means that all **training** data points must be correctly classified.
- ▶ This is ensured by the constraints, and can be regarded as a perceptron classifier.
- ▶ Thus, hard-margin SVM can be regarded as: **Choose a classifier from all possible perceptron classifiers that has the largest margin.**
- ▶ We motivated by that the most robust classifier is better ‘**regularized**’, which corresponds to the **weight decay** term in the objective function.

Optimization Method

$$\underset{\boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \|\boldsymbol{\theta}\|_2^2$$

$$\text{subject to} \quad y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n$$

- ▶ The learning problem is to minimize a (convex) **quadratic function** over **linear constraints**. In optimization society, this is a pretty much well developed area, called **quadratic programming**. There are plenty of solvers for quadratic programming, which are quite stable and fast. Thus, we can apply the well established solver for the hard-margin SVM. The details are out of the scope of the course.
- ▶ Additionally, solving hard-margin SVM formulation is rare in practice.

■

Is Max Margin Theoretically Better?

Regularization View and Modified VC Analysis

- ▶ As we said, we can relate the minimization of ℓ_2 -norm to the ℓ_2 -regularization, which in part explains it is better 'regularized' to tolerate noise.

Regularization View and Modified VC Analysis

- ▶ As we said, we can relate the minimization of ℓ_2 -norm to the ℓ_2 -regularization, which in part explains it is better 'regularized' to tolerate noise.

More rigorously, we can have the modified VC analysis for max margin hyperplane.

- ▶ The VC dimension of a hypothesis set is the maximum number of points that can be shattered. For any (binary) linear classifier, we have shown that it is $d + 1$.
- ▶ Consider the hypothesis set $\mathcal{H}_\rho$ containing all hyperplanes of margin at least ρ . Our goal is to show that restricting the hypothesis set to max margin classifier can decrease the number of points that can be shattered, that is, $d_{VC}(\rho) < d + 1$.

Modified VC Analysis

In general, we can prove the following result

Theorem: VC dimension of margin- ρ hyperplanes

Suppose the input space is the ball of radius R in $\mathbb{R}^d$, that is $\|\mathbf{x}\| \leq R$. Then,

$$d_{\text{VC}}(\rho) \leq \lceil R^2/\rho^2 \rceil + 1.$$

- ▶ Since its VC dimension is at most $d + 1$. Thus, we can have

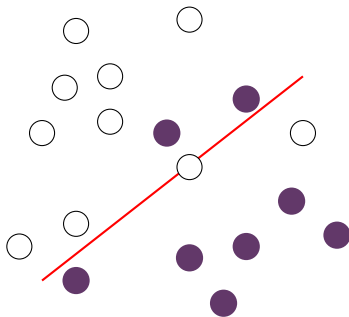
$$d_{\text{VC}}(\rho) \leq \min\{\lceil R^2/\rho^2 \rceil, d\} + 1.$$

- ▶ The bound suggests that the margin ρ can be used to control the model complexity. Hence, seeking for a **max margin** (**max ρ**) can lead to better Er_{out} .

▪

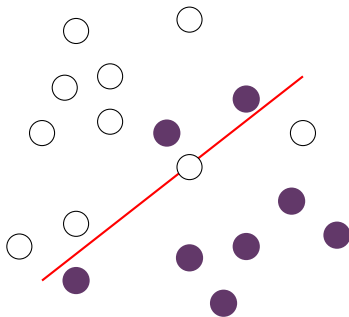
Soft-margin SVM

Non-linearly Separable Case



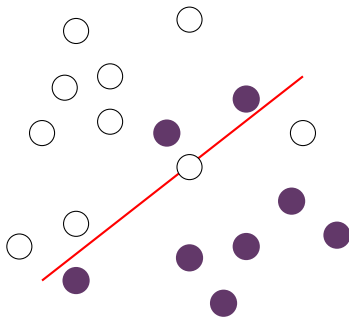
- ▶ A more practical case. For example, medium noise.
- ▶ Issue with hard-margin: some points will never satisfy the margin constraint.
- ▶ What will happen to the hard-margin SVM problem?

Non-linearly Separable Case



- ▶ A more practical case. For example, medium noise.
- ▶ Issue with hard-margin: some points will never satisfy the margin constraint.
- ▶ What will happen to the hard-margin SVM problem? Infeasible.
- ▶ How to solve this issue?

Non-linearly Separable Case



- ▶ A more practical case. For example, medium noise.
- ▶ Issue with hard-margin: some points will never satisfy the margin constraint.
- ▶ What will happen to the hard-margin SVM problem? Infeasible.
- ▶ How to solve this issue? Relaxation.

The Soft-margin SVM

We first introduce the **Hinge loss**:

$$h(\boldsymbol{\theta}; \mathbf{x}_i, y_i) = \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b))$$

- ▶ $h(\boldsymbol{\theta}) = 0$ if the point $\mathbf{x}_i$ lies in the correct side.
- ▶ For data on the wrong side, the function's value is proportional to the distance from the maximum-margin hyperplane.

The Soft-margin SVM

We first introduce the **Hinge loss**:

$$h(\boldsymbol{\theta}; \mathbf{x}_i, y_i) = \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b))$$

- ▶ $h(\boldsymbol{\theta}) = 0$ if the point $\mathbf{x}_i$ lies in the correct side.
- ▶ For data on the wrong side, the function's value is proportional to the distance from the maximum-margin hyperplane.

The **soft-margin** SVM is to solve

$$\underset{\boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)) + \lambda \|\boldsymbol{\theta}\|_2^2$$

- ▶ $\lambda > 0$ determines the trade-off between increasing the margin and ensuring that $\mathbf{x}_i$ lies on the correct side of the margin.
- ▶ If the data is linearly separable, then we can choose a sufficiently small λ to let the soft-margin SVM works the same as hard-margin SVM.
- ▶ If data is not linearly separable, soft-margin SVM can still provide a meaningful classifier.

Subgradient method for Soft-margin SVM

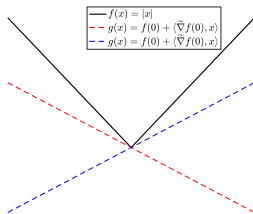
How to Solve The Soft-margin SVM?

- ▶ The hinge loss is **not differentiable**.
- ▶ One popular approach is to consider the **dual** formulation and then apply **coordinate algorithm** (in LibSVM). We consider another popular method, the **subgradient method**.

Subgradient: A generalization of gradient.

The subgradient of a **convex** $\mathcal{L}$ at θ is defined as any element of the following **subdifferential**

$$\partial \mathcal{L}(\theta) = \{s \in \mathbb{R}^d : \mathcal{L}(w) \geq \mathcal{L}(\theta) + s^\top (w - \theta), \quad \forall w \in \mathbb{R}^d\}$$



Subgradient Method

Soft-margin SVM:

$$\underset{\boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \mathcal{L}(\boldsymbol{\theta}) := \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)) + \lambda \|\boldsymbol{\theta}\|_2^2$$

Algorithmic design framework

$$\boldsymbol{\theta}_{k+1} = \underset{\boldsymbol{\theta} \in \Theta}{\operatorname{argmin}} \left\{ l_k(\boldsymbol{\theta}) = q_k(\boldsymbol{\theta}) + \frac{1}{2\mu_k} \|\boldsymbol{\theta} - \boldsymbol{\theta}_k\|_2^2 \right\}$$

- When $q_k(\boldsymbol{\theta})$ is linear approximation using subgradient $\implies$ **subgradient method**

Subgradient method

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \mathbf{s}_k, \quad \forall \mathbf{s}_k \in \partial \mathcal{L}(\boldsymbol{\theta}_k)$$

Stepsize in Subgradient Method

- ▶ Assume we are going to minimize

$$\min_{\theta} \mathcal{L}(\theta) = |\theta|$$

- ▶ The subdifferential of $|\theta|$ is $\partial\mathcal{L}(\theta) = \text{sign}(\theta)$.
- ▶ What will happen if we choose $\theta^0 = 0.05$ and $\mu_k = 0.1$?

Stepsize in Subgradient Method

- ▶ Assume we are going to minimize

$$\min_{\theta} \mathcal{L}(\theta) = |\theta|$$

- ▶ The subdifferential of $|\theta|$ is $\partial\mathcal{L}(\theta) = \text{sign}(\theta)$.
- ▶ What will happen if we choose $\theta^0 = 0.05$ and $\mu_k = 0.1$? It will never converge.
- ▶ This is due to the fact that subgradient method is a **non-descent method**.

Stepsize in Subgradient Method

- ▶ Assume we are going to minimize

$$\min_{\theta} \mathcal{L}(\theta) = |\theta|$$

- ▶ The subdifferential of $|\theta|$ is $\partial\mathcal{L}(\theta) = \text{sign}(\theta)$.
- ▶ What will happen if we choose $\theta^0 = 0.05$ and $\mu_k = 0.1$? It will never converge.
- ▶ This is due to the fact that subgradient method is a **non-descent method**.

Solution:

- ▶ The stepsize in subgradient method is often chosen as **diminishing**.
- ▶ **Diminishing stepsize** is a solution for ensuring convergence for non-descent method.

Two rule of thumb:

$$\mu_k = \frac{c}{k} \text{ or } \frac{c}{\sqrt{k}}, \quad \text{for some } c > 0.$$

Extension: SVM for Multi-class Classification

- ▶ SVM is intrinsically designed for binary classification. Unlike LR, its derivation cannot be generalized to multi-class case.
- ▶ The dominant approach for applying SVM to multi-class classification is to reduce the single multi-class problem into multiple binary classification problems.

Typical approaches include:

- ▶ One-verses-all: Classify one class from all other classes.
- ▶ One-versus-one: Classify every pair of classes.

↪ Next lecture: Kernel method.